# Requirement Understanding

Case Study: Community Library Membership & Book Lending System

Case Study Requirement

A community library wants a console-based application to manage:

- Members
- Books
- Book categories
- Book copies
- BorrowingwordMem
- Returns
- Fine calculation
- Membership limits

The system should use PostgreSQL as the database and EF Core for database access.

---

Core Functional Requirements

1. Member Management

The system should allow:

- Add a new member
- View all members
- Search member by phone number or email
- Update membership status
- Deactivate a member

Each member should have a membership type such as:

- Basic
- Premium
- Student

---

2. Book Management

The system should allow:

- Add a new book
- Add multiple copies of a book
- View available books
- Search books by title, author, or category
- Mark a book copy as damaged or unavailable

---

3. Borrowing Rules

The system must apply business logic before allowing a member to borrow a book.

Business Logic Criteria

| Membership Type | Maximum Active Borrowings | Maximum Borrow Days |
|---|---|---|
| Basic | 2 books | 7 days |
| Student | 3 books | 10 days |
| Premium | 5 books | 15 days |

A member cannot borrow a book if:

- Their membership is inactive
- They already reached the borrowing limit
- The book copy is not available
- They have unpaid fines above ₹500
- They already borrowed the same book and have not returned it

---

4. Return Rules

When a book is returned:

- The system should calculate late return fine

- Fine amount = ₹10 per delayed day
- The returned book copy should become available again
- Borrowing status should change to returned
- Return date should be saved

---

5. Fine Management

The system should allow:

- View pending fines of a member
- Pay fine
- View fine history
- Prevent borrowing if unpaid fine is above ₹500

---

Stored Procedure / PostgreSQL Function Requirement

Participants must create at least one PostgreSQL function and call it from EF Core.

Suggested functions:

1. calculate_member_fine(member_id)

o Returns total unpaid fine for a member

2. get_available_books_by_category(category_id)

o Returns available books under a category

3. get_member_borrowing_summary(member_id)

o Returns active borrowed books, returned books, and total fine

---

Transaction Requirement

Participants must use EF Core transaction for the borrowing process.

Borrowing a book should happen as one transaction:

1. Validate member

2. Validate unpaid fine
3. Check active borrowing count
4. Check book copy availability
5. Create borrowing record
6. Update book copy status as borrowed
7. Commit transaction

If any step fails, rollback the transaction.

---

Suggested Console Menu

1. Member Management
2. Book Management
3. Borrow Book
4. Return Book
5. Fine Management
6. Reports
7. Exit

---

Reports

Add report options such as:

- Books currently borrowed
- Overdue books
- Members with pending fines
- Most borrowed books
- Available books by category
- Member borrowing history

---

DataBase Design

## Database Design

### 1. membershipTypes

| Attribute | Data Type |
|---|---|
| MembershipTypeId | int |
| Name | text |
| MaxBooksAllowed | int |
| MaxBorrowDays | int |

### 2. members

| Attribute | Data Type |
|---|---|
| MemberId | int |
| Name | text |
| Email | text |
| Phone | text |
| MembershipTypeId | int |
| IsActive | boolean |
| CreatedAt | timestamp with time zone |

### 3. bookCategories

| Attribute | Data Type |
|---|---|
| BookCategoryId | int |
| CategoryName | text |

### 4. books

| Attribute | Data Type |
|---|---|
| BookId | int |
| Title | text |
| Author | text |
| BookCategoryId | int |
| PublishedYear | int |

### 5. bookCopies

| Attribute | Data Type |
|---|---|
| BookCopyId | int |
| BookId | int |
| CopyCode | text |

| Attribute | Data Type |
| --- | --- |
| Status | text |

**6. borrowings**

| Attribute | Data Type |
| --- | --- |
| BorrowingId | int |
| MemberId | int |
| BookCopyId | int |
| BorrowDate | timestamp with time zone |
| DueDate | timestamp with time zone |
| ReturnDate | timestamp with time zone |
| Status | text |

**7. fines**

| Attribute | Data Type |
| --- | --- |
| FineId | int |
| BorrowingId | int |
| Amount | decimal |

| Attribute | Data Type |
| --- | --- |
| IsPaid | boolean |

**8. finePayments**

| Attribute | Data Type |
| --- | --- |
| FinePaymentId | int |
| FineId | int |
| AmountPaid | decimal |
| PaymentDate | timestamp with time zone |

Procedures

1. Add Category Procedure

```
-- PROCEDURE: public.add_category(text)

-- DROP PROCEDURE IF EXISTS public.add_category(text);

CREATE OR REPLACE PROCEDURE public.add_category(
	IN p_categoryname text)
LANGUAGE 'plpgsql'
AS $BODY$
BEGIN

   INSERT INTO "bookCategories"
   ("CategoryName")
   VALUES
   (p_categoryname);

END;
$BODY$;
ALTER PROCEDURE public.add_category(text)
   OWNER TO postgres;
```

2. Add Fine Procedure

```
 -- PROCEDURE: public.add_fine(integer, numeric)

-- DROP PROCEDURE IF EXISTS public.add_fine(integer, numeric);

CREATE OR REPLACE PROCEDURE public.add_fine(
	IN p_borrowingid integer,
```

```
        IN p_amount numeric)
LANGUAGE 'plpgsql'
AS $BODY$
BEGIN

   INSERT INTO "fines"
   (
      "BorrowingId",
      "Amount",
      "IsPaid"
   )
   VALUES
   (
      p_borrowingid,
      p_amount,
      false
   );

END;
$BODY$;
ALTER PROCEDURE public.add_fine(integer, numeric)
   OWNER TO postgres;
```

3.Return Book

```
-- PROCEDURE: public.return_book(integer, timestamp with time zone, text)

-- DROP PROCEDURE IF EXISTS public.return_book(integer, timestamp with time zone, text);

CREATE OR REPLACE PROCEDURE public.return_book(
```

```
        IN p_borrowingid integer,
        IN p_returndate timestamp with time zone,
        IN p_status text)
LANGUAGE 'plpgsql'
AS $BODY$
BEGIN

   UPDATE "borrowings"
   SET
      "ReturnDate" = p_returndate,
      "Status" = p_status
   WHERE "BorrowingId" = p_borrowingid;

END;
$BODY$;
ALTER PROCEDURE public.return_book(integer, timestamp with time zone, text)
   OWNER TO postgres;
```

4. update book copy status procedure

```
-- PROCEDURE: public.update_book_copy_status(integer, text)

-- DROP PROCEDURE IF EXISTS public.update_book_copy_status(integer, text);

CREATE OR REPLACE PROCEDURE public.update_book_copy_status(
        IN p_copyid integer,
        IN p_status text)
LANGUAGE 'plpgsql'
AS $BODY$
BEGIN
```

```
    UPDATE "bookCopies"
    SET "Status" = p_status
    WHERE "BookCopyId" = p_copyid;

END;
$BODY$;
ALTER PROCEDURE public.update_book_copy_status(integer, text)
    OWNER TO postgres;
```

# ScreenShots

```
===== Library Management System =====
1. Member Management
2. Book Management
3. Borrow Book
4. Return Book
5. Fine Management
6. Reports
7. Exit

Enter Choice : 1
```

Fig1: Displaying the menu of the Library Management System

```
--- Member Management ---
1.Add a Member
2.View All Members
3.Search Member by Phone
4.Search Member by Email
5.Update Membership Status
6.Deactivate a Member

Enter Choice : 2

-----------------------
Id : 1
Name : Teenu Deekshith
Email : teenudeekshith982@gmail.com
Phone : 9392266077
Membership : Student
Active : False

-----------------------
Id : 2
Name : Vikram
Email : vikram123@gmail.com
Phone : 8341523460
```

Fig2: Displaying all the Members in the Library

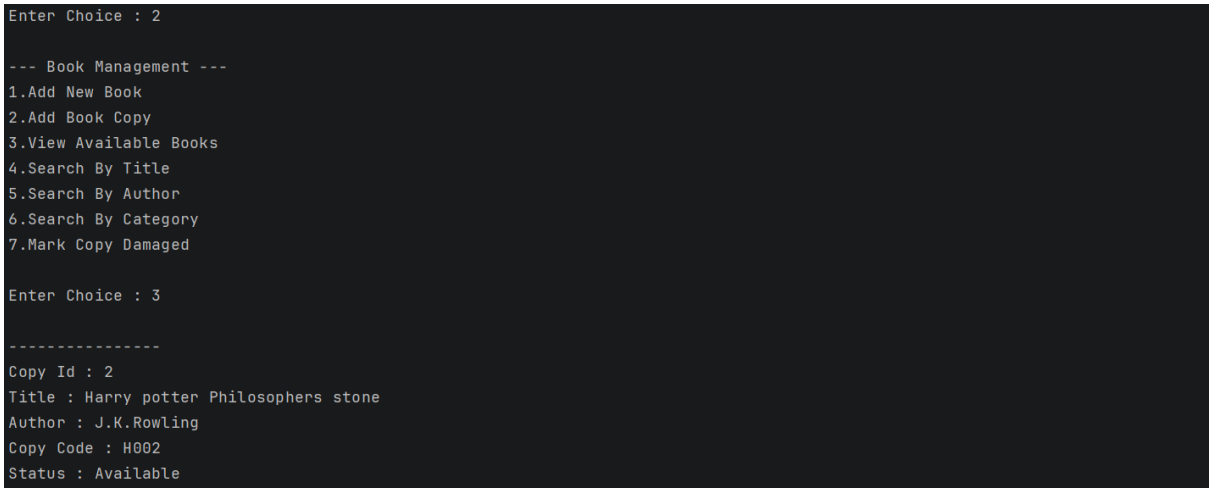


Fig3 : Displaying the menu in the book management system and viewing the available books.


Fig 4: Borrowing book Function

```
===== Library Management System =====
1. Member Management
2. Book Management
3. Borrow Book
4. Return Book
5. Fine Management
6. Reports
7. Exit

Enter Choice : 5

--- Fine Management ---
1.View Pending Fines
2.Pay Fine
3.View Fine Payment History

Enter Choice : 1
Enter Member Id : 2
No pending fines
```

Fig5 : Fine Management

```
--- Reports Module ---
1.Books Currently Borrowed
2.Overdue Books
3.Members With Pending Fines
4.Most Borrowed Books
5.Available Books By Category
6.Member Borrowing History

Enter Choice : 1

---------------
Borrowing Id : 2
Member : Vikram
Book : Five point someone
Borrow Date : 18-05-2026 17:51:33
Due Date : 02-06-2026 17:51:33

---------------
```

Fig 6: Books Currently Borrowed


Fig 7: Most Borrowed Books

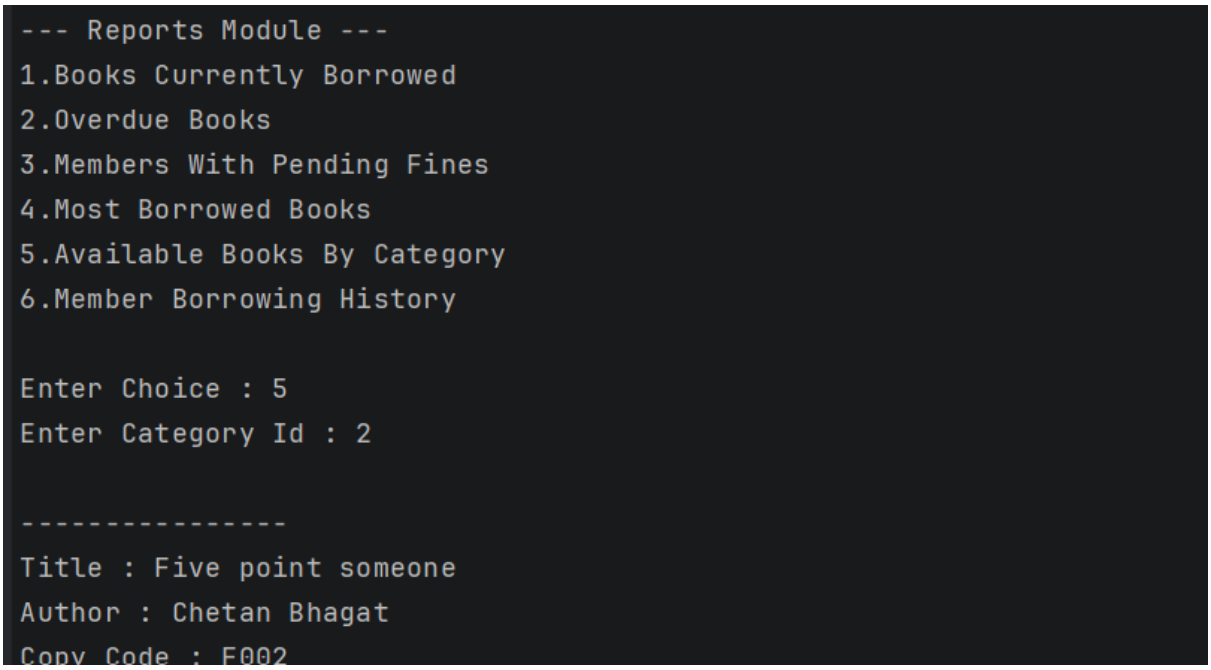


Fig8 : Available Books By category

```
===== Library Management System =====
1. Member Management
2. Book Management
3. Borrow Book
4. Return Book
5. Fine Management
6. Reports
7. Exit

Enter Choice : 4

--- Return Book ---
Enter Borrowing Id : 2
Book returned successfully
```

Fig 9: Returning a book

```
1.Books Currently Borrowed
2.Overdue Books
3.Members With Pending Fines
4.Most Borrowed Books
5.Available Books By Category
6.Member Borrowing History

Enter Choice : 6
Enter Member Id : 2

----------------
Book : Harry potter Philosophers stone
Borrow Date : 18-05-2026 17:57:00
Return Date :
Status : Borrowed

----------------
Book : Five point someone
Borrow Date : 18-05-2026 17:51:33
Return Date : 18-05-2026 17:58:31
Status : Returned
```

Fig 10: Member browsing History